

Jacobian Space from Scratch: Qwen3.6-27B

Anthropic 提出了 **Jacobian lens**,主張它能讀出語言模型「還沒說出口的想法」。這篇用 MLX 在 Apple Silicon 上從零實作一次,看它到底在算什麼。

走法:

1. 先認識 Qwen3.6-27B 的架構,走一次 forward
2. 做最偷懶的 **logit lens**,看它哪裡不夠好
3. 換成 **Jacobian lens**,把 logit lens 偷懶的地方補回來
4. 一個有趣的例子

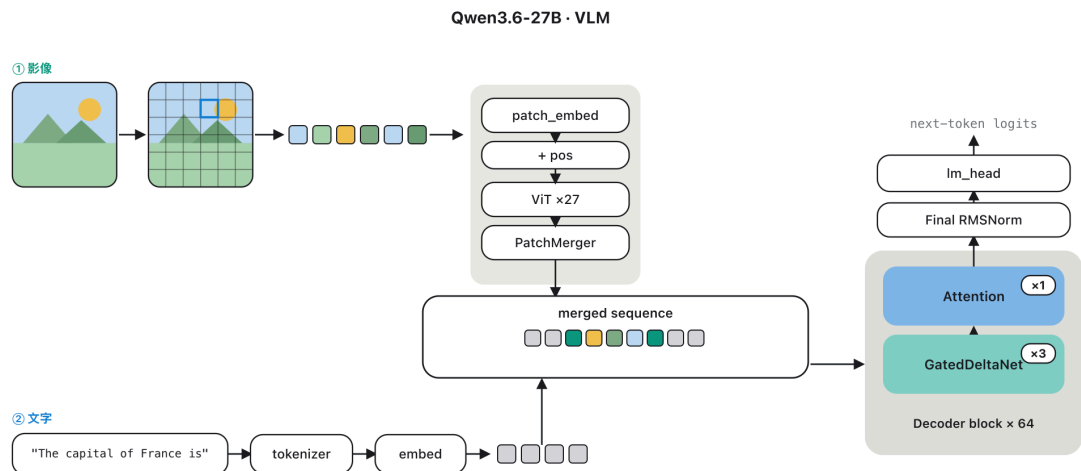
模型放在 `models/Qwen3.6-27B-4bit` ([mlx-community 版](#)), 跑在 `mlx-vlm 0.6.4`。

```
In [1]: from mlx_vlm import load
        from rich import print

def patch_qwen3_5():
    """mlx-vlm #1548: 0.6.4 misses the +1.0 shift on qwen3_5 RMSNorm
    weights,
    which garbles every output. Must run before load()."""
    import mlx_vlm.models.qwen3_5.qwen3_5 as _q
    keys = (".input_layernorm.weight", ".post_attention_layernorm.weight",
            "model.norm.weight", ".q_norm.weight", ".k_norm.weight")
    if getattr(_q.Model, "sanitize", "_patched", False):
        return
    base = _q.sanitize_key
    def sanitize(self, weights):
        shift = any("mtp." in k for k in weights) or any(
            "conv1d.weight" in k and v.shape[-1] != 1 for k, v in
weights.items())
        weights = {k: v for k, v in weights.items() if "mtp." not in k}
        if self.config.text_config.tie_word_embeddings:
            weights.pop("lm_head.weight", None)
        out = {}
        for k, v in weights.items():
            k = base(k)
            if "conv1d.weight" in k and v.shape[-1] != 1:
                v = v.moveaxis(2, 1)
            if shift and any(k.endswith(s) for s in keys) and v.ndim == 1:
                v = v + 1.0
            out[k] = v
        return out
    sanitize._patched = True
    _q.Model.sanitize = sanitize

patch_qwen3_5()
```

1. Qwen3.6-27B 的架構



這是多模態模型,圖和文字各走一條前處理,最後併成同一串 sequence 送進 decoder。

文字:tokenizer 把字串切成 token id,再去 embedding 表查出 5120 維向量。

影像:切成 16x16 的 patch,每個 patch 過 vision encoder (ViT) 編碼,再由 PatchMerger 把 2x2 個 patch 併成一個、投影到 5120 維。Qwen 的 vision encoder 設計上接近 SigLIP-2 那一系。

兩邊出來的向量都是 5120 維,併成同一串 sequence 之後,decoder 就不分誰是圖誰是字了。最後一個位置的輸出過 `lm_head`,變成 248,320 個候選字的分數,取最高分就是下一個 token。

經典論文: [Attention Is All You Need](#) (Transformer 本體) · [An Image is Worth 16x16 Words](#) (ViT) · [SigLIP 2](#) (vision encoder) · [Qwen2-VL](#) (圖文怎麼併、MRoPE 怎麼標位置)

```
In [2]: model, processor = load("models/Qwen3.6-27B-4bit")
        tokenizer = processor.tokenizer

        llm = model.language_model
        inner = llm.model
        cfg = model.config.text_config
```

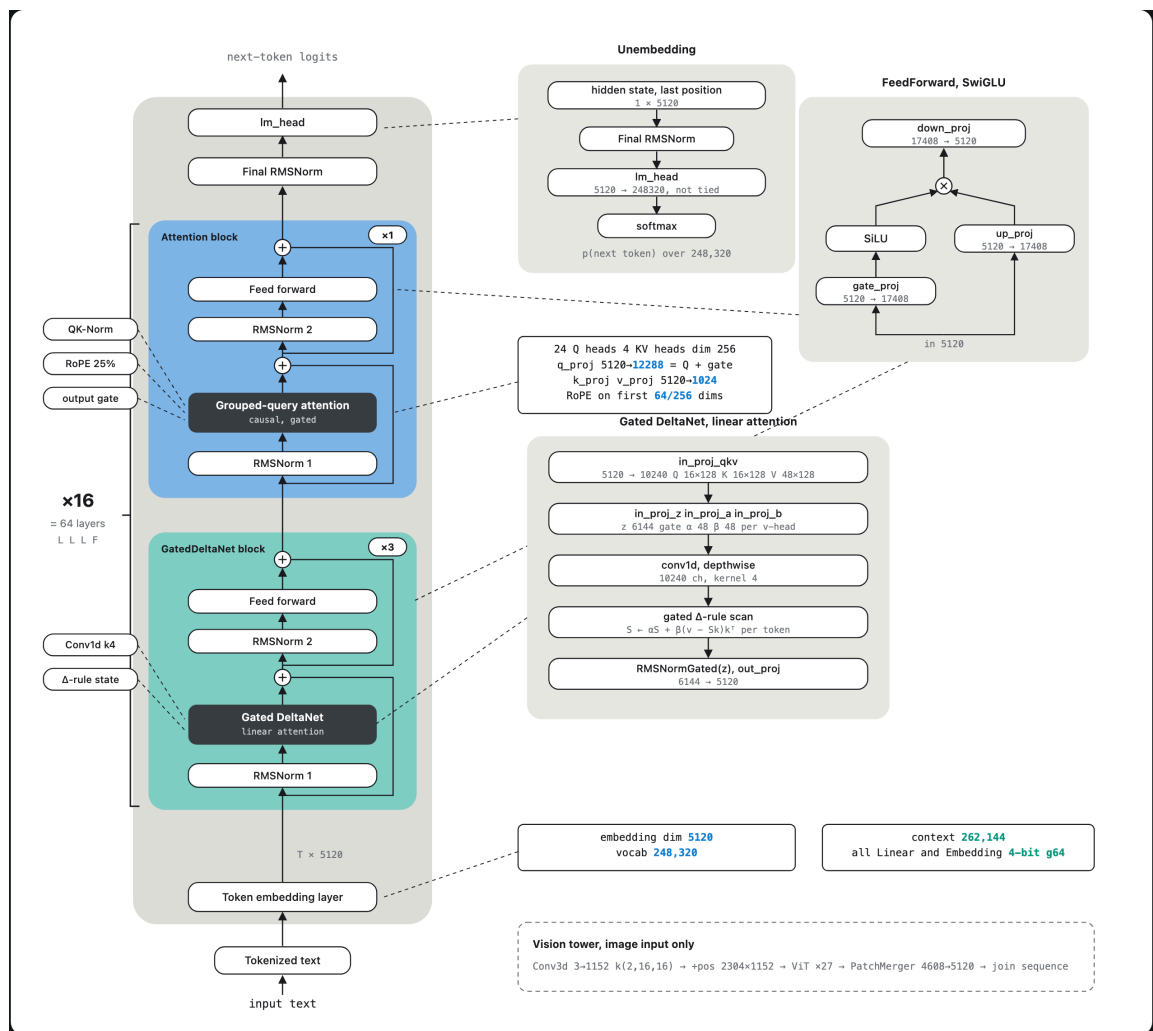
64 層由兩種 block 疊成,每 4 層一組。

```
In [3]: for i, layer in enumerate(inner.layers[:4]):
        kind = "GatedDeltaNet (linear attn)" if layer.is_linear else
        "Attention (full attn)"
        print(f"layer {i:2d}    {kind}")
        print(f"    ...      same pattern x 16  ->  {len(inner.layers)} layers
        total")
```

```
layer 0    GatedDeltaNet (linear attn)
layer 1    GatedDeltaNet (linear attn)
layer 2    GatedDeltaNet (linear attn)
layer 3    Attention (full attn)
```

... same pattern x 16 -> 64 layers total

兩種 decoder block



兩種 block 的外框一模一樣,都是在 residual stream 上加東西:

```
h <- h + mixer(RMSNorm(h))
h <- h + FFN(RMSNorm(h))
```

FFN 是 SwiGLU, 5120 升到 17408 再壓回 5120。差別只在中間那個 **mixer** 怎麼看前文。

Full attention block

就是標準 causal attention。Qwen 的設定: 24 個 Q head、4 個 KV head (GQA, 6 個 Q 共用一組 KV), head_dim 256。Q 和 K 各自先過 RMSNorm (QK-Norm) 穩住數值, RoPE 只轉前 64 維 (256 的 25%), 而且用 MRoPE 三軸編位置, 這樣圖片 patch 才能標上二維座標。

有一個地方跟課本版不同。 q_proj 的輸出是 $24 \times 256 \times 2 = 12288$ 維, 一半當 Q, 另一半當 **output gate**:

```
attn = softmax(Q K^T / sqrt(256)) V
out = o_proj(attn * sigmoid(gate))
```

attention 算完之後先被 sigmoid gate 逐維縮放, 才進 o_proj (6144 → 5120) 寫回 residual stream。gate 讓這一層自己決定這次要寫多少進去。

GatedDeltaNet block

這個少見,講細一點。它不回頭看整串 sequence,而是維護一個固定大小的 **state**,邊讀邊更新。

每個 head 的 state 是一個 `128 x 128` 矩陣 `S` (value 維 x key 維)。可以想成一本查找表: `k` 是索引, `v` 是內容。

每讀一個 token 做四件事:

```
S <- g * S           # decay: 舊資訊按比例衰減
d <- beta * (v - S k) # delta rule: 新 value 減掉查表結果
S <- S + d k^T        # write: 只把差額寫回去
y <- S q             # read: 用 q 查表,得到這個位置的輸出
```

關鍵是中間兩步的 **delta rule**。它不硬把 `v` 塞進表裡,而是先用 `k` 查一次現在表裡有什麼 (`S k`),算出差額 `v - S k`,只補這個差額。所以同一個 key 被重複寫入時是覆蓋,不是疊加。

兩個 gate 控制力道,每個 head 各一個純量:

```
g = exp(-exp(A_log) * softplus(a + dt_bias)) # decay
gate, 落在 (0,1)
beta = sigmoid(b)                             # write
gate
```

`a`、`b` 都是從當前 token 算出來的 (`in_proj_a`、`in_proj_b`, 5120 -> 48),
`A_log`、`dt_bias` 是學來的參數。所以「忘多少、寫多強」是看內容決定,不是固定值。

進遞迴前還有一步: `q`、`k`、`v` 先過 depthwise causal conv1d (kernel 4) 加 SiLU,讓相鄰幾個 token 先混一下, `q` 和 `k` 再各自 normalize 並縮放。出來之後過一個 gated RMSNorm (gate 由 `in_proj_z` 給),最後 `out_proj` (6144 -> 5120) 寫回 residual stream。

Qwen 的設定: 16 個 `q/k` head、48 個 `v` head, `head_dim` 都是 128。所以一層的 state 是 `48 x 128 x 128`, 約 79 萬個數字,跟 **sequence 多長完全無關**。

經典論文: [RMSNorm](#) · [SwiGLU](#) · [RoPE](#) · [GQA](#) · [Linear Transformers](#) (linear attention 起點) · [Fast Weight Programmers](#) (delta rule 進到這一系) · [Gated Delta Networks](#) (這個 block 本身) · [Mamba-2](#) (同一系的 state space 做法)

兩種 mixer 的記憶成本差在哪,算一下就清楚了。

```
In [4]: attn = inner.layers[3].self_attn
        gdn = inner.layers[0].linear_attn

        kv_per_token = 2 * attn.num_key_value_heads * attn.head_dim
        state_size = gdn.num_v_heads * gdn.head_v_dim * gdn.head_k_dim

        print(f"full attention    {attn.num_attention_heads} Q heads /
        {attn.num_key_value_heads} KV heads x {attn.head_dim}")
        print(f"                KV cache grows: {kv_per_token} numbers per token
        per layer")
        print(f"GatedDeltaNet    {gdn.num_v_heads} v heads x {gdn.head_v_dim} x
        {gdn.head_k_dim} state")
        print(f"                fixed: {state_size:,} numbers per layer, any
        sequence length")
        print(f"\nbreak-even at {state_size // kv_per_token} tokens; context limit
        is {cfg.max_position_embeddings:,}")
```

```
full attention    24 Q heads / 4 KV heads x 256
                  KV cache grows: 2048 numbers per token per layer
GatedDeltaNet    48 v heads x 128 x 128 state
                  fixed: 786,432 numbers per layer, any sequence
length
break-even at 384 tokens; context limit is 262,144
```

所以 3:1 的用意很直接:大部分層用便宜的壓縮記憶,每 4 層插一次 full attention 精準回頭撈細節。

輸入長什麼樣

文字和影像各跑一次前處理,看實際的形狀。

```
In [5]: from PIL import Image

        text = "The capital of France is"
        text_ids = tokenizer.encode(text)

        print("TEXT")
        print(f"  input      : {text!r}")
        print(f"  token ids  : {text_ids}")
        print(f"  tokens     : {[tokenizer.decode([i]) for i in text_ids]}")

        img = Image.open("assets/qwen3_6_27b_arch_residual.png")
        vision = processor.image_processor(images=img, return_tensors="np")
        pixel_values, grid = vision["pixel_values"], vision["image_grid_thw"]
        t, h, w = grid[0]
        merge = processor.image_processor.merge_size

        print("\nIMAGE")
        print(f"  file       : {img.size[0]}x{img.size[1]} px")
        print(f"  grid t,h,w : {t},{h},{w} -> {t*h*w} patches")
        print(f"  pixel_values: {pixel_values.shape}    1536 = temporal 2 x RGB 3 x
        16 x 16")
        print(f"  after {merge}x{merge} merge: {(h//merge)*(w//merge)} vectors
        into the decoder")
```

TEXT

```
input      : 'The capital of France is'
token ids  : [760, 6511, 314, 9338, 369]
tokens     : ['The', ' capital', ' of', ' France', ' is']
```

IMAGE

```
file       : 1660x2260 px
grid t,h,w : 1,142,104 -> 14768 patches
pixel_values: (14768, 1536) 1536 = temporal 2 x RGB 3 x 16 x 16
after 2x2 merge: 3692 vectors into the decoder
```

五個字是 5 個 token,一張圖攤開卻是上萬個 patch,併完還剩幾千個向量。圖片吃 context 吃得很兇,這也是為什麼 merge 那一步要再壓一次。

走一次 forward,確認模型正常。

```
In [6]: import mlx.core as mx

ids = mx.array(tokenizer.encode("The capital of France is"))[None]
logits = llm(ids).logits
next_id = int(mx.argmax(logits[0, -1]))

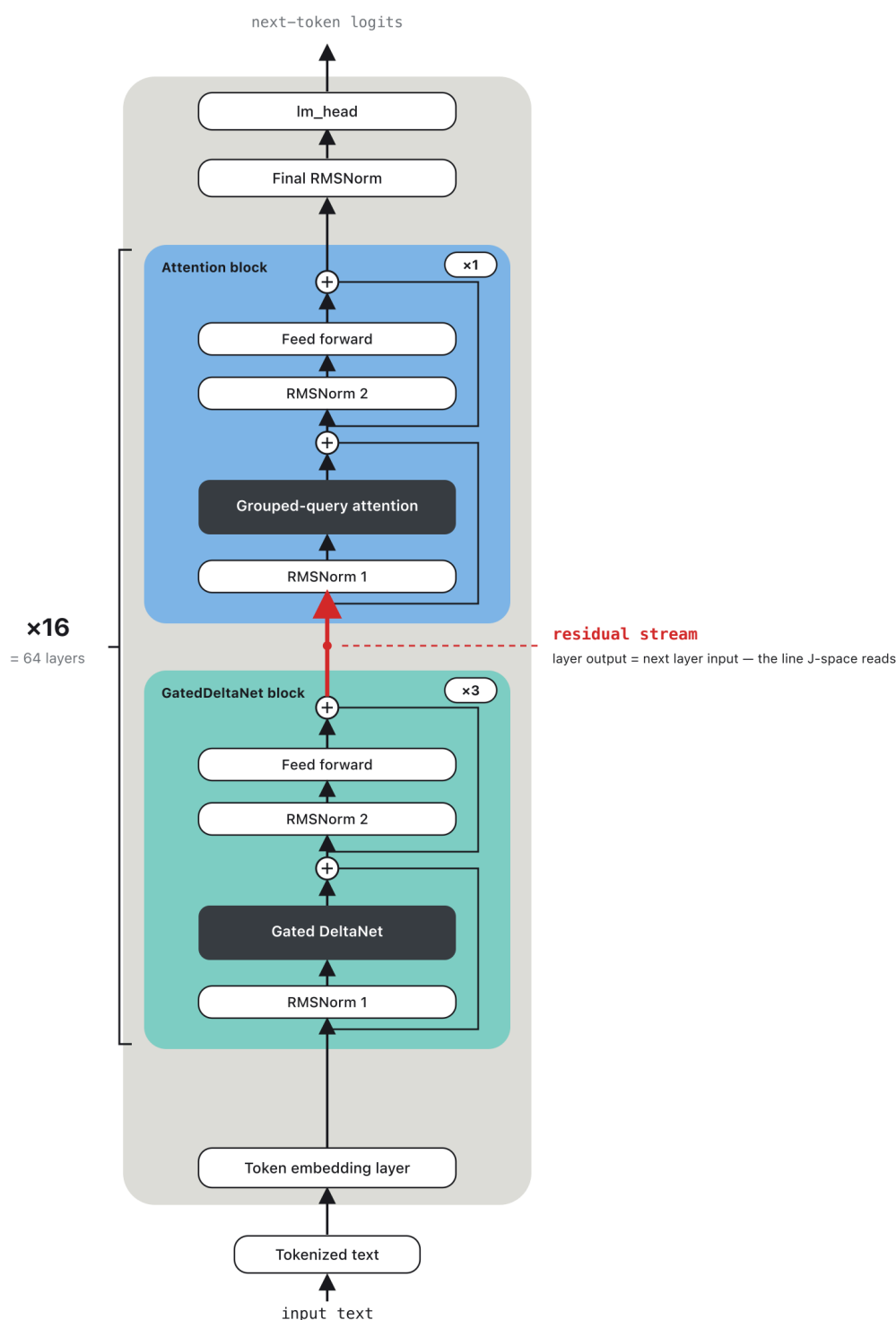
print(f"logits      : {logits.shape}")
print(f"next token  : {tokenizer.decode([next_id])!r}")
```

```
logits      : (1, 5, 248320)
next token  : ' Paris'
```

2. Logit lens

可解釋性 (interpretability) 想問的是:模型吐出答案的路上,中間到底發生了什麼。

Transformer 有個很好的切入點:**residual stream**。每一層不是重寫整條向量,而是把自己算的東西加回同一條向量上。這條貫穿 64 層的線就是模型的工作記憶,而且因為只有加法,每一層的輸出都停在**同一個座標系裡** ([A Mathematical Framework for Transformer Circuits](#), Elhage et al. 2021)。



既然每層輸出都在同一個座標系上,那能不能直接拿尺去量?

這就是 **logit lens** 的想法 ([Interpreting GPT: the logit lens](#), nostalgebraist 2020)。模型最後那顆頭本來只讀最終的 h_L , 現在把它借來, 對**每一層**的 h_ℓ 都讀一次:

$$\text{LogitLens}(h_\ell) = W_U \cdot \text{norm}(h_\ell)$$

norm 是 final RMSNorm, W_U 是 `lm_head`。讀出來的就是「第 ℓ 層時, 模型以為下一個字是什麼」。

先把 residual 抽出來。這裡要手動一層層跑, 自己準備兩種 mask 和 MRoPE 位置, 這樣才跟官方 forward 完全一致。

```

In [7]: from mlx_vlm.models.qwen3_5.language import (
        _create_qwen3_5_attention_mask, _create_qwen3_5_ssm_mask)

dtype = inner.norm.weight.dtype
d_model = cfg.hidden_size
n_layers = len(inner.layers)

def encode(text):
    return mx.array(tokenizer.encode(text))[None]

def prep(h):
    fa = _create_qwen3_5_attention_mask(h, None)
    ssm = _create_qwen3_5_ssm_mask(h, None)
    pos = mx.tile(mx.arange(h.shape[1])[None, None, :], (3, 1, 1))
    return fa, ssm, pos

def set_linear_train(mode):
    """eval uses a non-differentiable metal kernel; train uses pure ops
    that mx.vjp
    can backprop through. Needed later for the Jacobian."""
    for layer in inner.layers:
        if layer.is_linear:
            layer.linear_attn.train(mode)

def residuals(ids):
    set_linear_train(False)
    h = inner.embed_tokens(ids)
    fa, ssm, pos = prep(h)
    out = []
    for layer in inner.layers:
        h = layer(h, mask=(ssm if layer.is_linear else fa),
                  cache=None, position_ids=pos, position_embeddings=None)
        out.append(h)
    return out

def readout(h):
    return llm.lm_head(inner.norm(h))

res = residuals(encode("The capital of France is"))
print(f"{len(res)} residuals, each {tuple(res[0].shape)} = (batch, tokens,
d_model)")

```

64 residuals, each (1, 5, 5120) = (batch, tokens, d_model)

```
In [8]: prompt = "The capital of France is"
ids = encode(prompt)
res = residuals(ids)
last = ids.shape[1] - 1

answer = int(mx.argmax(readout(res[-1])[0, last]))
print(f"prompt = {prompt!r}    answer = {tokenizer.decode([answer])!r}\n")

print("layer    top-1                p(top1)    p(answer)")
for l in range(0, n_layers, 4):
    logits = readout(res[l])[0, last].astype(mx.float32)
    probs = mx.softmax(logits)
    top = int(mx.argmax(logits))
    print(f"L{l:2d}    {tokenizer.decode([top])!r:18s}
{float(probs[top]):7.3f}    {float(probs[answer]):7.3f}")

prompt = 'The capital of France is'    answer = ' Paris'
```

layer	top-1	p(top1)	p(answer)
L 0	' '	0.010	0.000
L 4	' '	0.013	0.000
L 8	'...'	0.008	0.000
L12	'...'	0.005	0.000
L16	'...'	0.008	0.000
L20	'陷'	0.006	0.000
L24	'...'	0.006	0.000
L28	'_____'	0.008	0.000
L32	'zł'	0.007	0.000
L36	'_____'	0.032	0.000
L40	'...'	0.029	0.000
L44	'...'	0.070	0.000
L48	'_____'	0.012	0.000
L52	'_____'	0.023	0.000
L56	' Paris'	0.225	0.225
L60	' Paris'	0.858	0.858

軌跡亮得很晚。前 50 幾層讀出來全是 '...'、'_____' 這種碎片, $p(\text{answer})$ 幾乎是 0, 一直到 L56 才冒出 ' Paris', L60 才衝到 0.86。

但這是 **logit lens** 的問題, 不一定是模型的實情。把 h_ℓ 直接丟進最後那顆頭, 等於偷偷假設「從第 ℓ 層到終點的那幾十層什麼都不做」:

$$\text{LogitLens}(h_\ell) = W_U \cdot \text{norm}(h_\ell) \iff \frac{\partial h_L}{\partial h_\ell} \approx I$$

對最後幾層還堪用, 對中層就太粗了。答案可能早就算好, 只是還沒轉到最後那顆頭讀得到的方向上。

3. Jacobian lens

Anthropic 的做法很直接:那個被隨手當成 I 的東西,把它真的算出來。

$$J_\ell = \frac{\partial h_L}{\partial h_\ell} \quad \text{JLens}(h_\ell) = W_U \cdot \text{norm}(J_\ell h_\ell)$$

J_ℓ 是「第 ℓ 層的 residual 動一點,最終的 residual 會怎麼跟著動」的線性近似, 一個 5120×5120 的矩陣。乘上它,等於先把 h_ℓ 搬到最終 residual 的座標系, 再用同一顆頭去讀。跟 logit lens 的差別只有 $\times J_\ell$ 這一步。

官方實作:[anthropics/jacobian-lens](https://anthropics.com/jacobian-lens)。社群解說:[J-Space: Yet Another LLM Mind Reader?](#)。一份獨立重現與批評:[A Review of Anthropic's Global Workspace Paper](#)。

從零算 J_ℓ

要拿到 J_ℓ , 用 `mx.vjp` 反傳最直接:在第 `src` 層之後注入一個 delta, 看第 `tgt` 層的 residual 怎麼變。

有兩個實務細節:

1. GatedDeltaNet 在 eval 模式走的是不可微的 metal kernel,所以算之前要切到 train 模式 (純 ops 的 chunked scan),算完切回來。
2. chunked scan 內部會呼叫 `mx.async_eval`, 那在 `mx.vjp` 裡不合法,暫時換成 `no-op`。

矩陣有 5120 列,全算太慢,這裡只算前 64 列證明它是真的算得出來。

```

In [9]: import contextlib
import time

SKIP_FIRST = 16  # early positions are BOS and formatting, readout there
is noise
TARGET = n_layers - 1

@contextlib.contextmanager
def no_async_eval():
    orig = mx.async_eval
    mx.async_eval = lambda *a, **k: None
    try:
        yield
    finally:
        mx.async_eval = orig

def forward_with_injection(ids, delta, src, tgt):
    h = inner.embed_tokens(ids)
    fa, ssm, pos = prep(h)
    for l, layer in enumerate(inner.layers):
        h = layer(h, mask=(ssm if layer.is_linear else fa),
                    cache=None, position_ids=pos, position_embeddings=None)
        if l == src:
            h = h + delta
        if l == tgt:
            return h
    return h

def jacobian_rows(ids, src, tgt, n_rows):
    seq = ids.shape[1]
    valid = mx.array(list(range(SKIP_FIRST, seq - 1)), dtype=mx.int32)
    delta = mx.zeros((n_rows, seq, d_model), dtype=dtype)
    onehot = (mx.arange(d_model)[None, :] == mx.arange(n_rows)[:,
None]).astype(mx.float32)
    posmask = (mx.arange(seq)[None, :] == valid[:, None]).any(axis=0)
    cot = (onehot[:, None, :] * posmask[None, :,
None]).astype(mx.float32).astype(dtype)

    set_linear_train(True)
    with no_async_eval():
        (_,), (g,) = mx.vjp(
            lambda d: forward_with_injection(ids, d, src, tgt), (delta,),
(cot,))
        rows = mx.take(g, valid, axis=1).mean(axis=1).astype(mx.float32)
        mx.eval(rows)
    set_linear_train(False)
    return rows

long_prompt = ("The history of the Roman Empire spans several centuries and
includes "
               "many emperors, wars, and cultural achievements that shaped
the ancient world.")
LAYER = 54

t0 = time.perf_counter()

```

```
rows = jacobian_rows(mx.array(tokenizer.encode(long_prompt)[:48])[None],
                      src=LAYER, tgt=TARGET, n_rows=64)
print(f"J[L{LAYER}] first 64 rows: {tuple(rows.shape)} in
{time.perf_counter()-t0:.0f}s")
print(f"diagonal entry J[0,0] = {float(rows[0, 0]):.3f}    row norm =
{float(mx.linalg.norm(rows[0])):.3f}")
```

J[L54] first 64 rows: (64, 5120) in 20s

diagonal entry J[0,0] = 0.949 row norm = 1.136

對角線大約 0.95, 不是 1。也就是說 logit lens 假設的 $J_\ell = I$ 差得不算天遠, 但差的那一點就是全部的關鍵。

換成公開的 lens

上面是用一條 prompt 算的。Anthropic 的做法是對很多條 prompt 取平均, 得到一個不綁特定輸入的 lens。這裡直接載 Neuronpedia 放出來的擬合結果 ([neuronpedia/jacobian-lens](https://neuronpedia.com/jacobian-lens)), 順手比對一下我們從零算的那 64 列跟它像不像。

```
In [10]: _lens = mx.load("models/Qwen3.6-27B-4bit/jlens.npz")
src_layers = [int(x) for x in _lens["__source_layers__"].tolist()]
n_prompts = int(_lens["__n_prompts__"].tolist()[0])
Jlens = {l: _lens[f"J_{l}"] for l in src_layers}

print(f"public lens: layers {src_layers[0]}..{src_layers[-1]}
({len(src_layers)}), "
      f"fitted on {n_prompts} prompts")

a = rows.reshape(-1)
b = Jlens[LAYER].astype(mx.float32)[:64].reshape(-1)
cos = float((a @ b) / (mx.linalg.norm(a) * mx.linalg.norm(b)))
print(f"cos(our 1-prompt rows, public {n_prompts}-prompt lens) =
{cos:.3f}")

def transport(h, l):
    """Move a layer-l residual into the final-residual basis: J_l @ h."""
    J = Jlens[l]
    return (h.astype(J.dtype) @ J.T).astype(dtype)
```

public lens: layers 0..62 (63), fitted on 1000 prompts

cos(our 1-prompt rows, public 1000-prompt lens) = 0.894

一條 prompt 算出來的東西就已經跟一千條擬合的 lens 高度對齊, 說明 J_ℓ 抓到的 是不太依賴輸入的結構。

兩把尺並排

同一次 forward, 同一顆頭, 唯一差別是有沒有乘 J_ℓ 。

```
In [11]: prompt = "The capital of France is"
ids = encode(prompt)
res = residuals(ids)
last = ids.shape[1] - 1
answer = int(mx.argmax(readout(res[-1])[0, last]))

print(f"{prompt!r} -> {tokenizer.decode([answer])!r}\n")
print("layer    logit p(ans)    jacobian p(ans)    jacobian top-1")
for l in range(48, n_layers):
    if l not in Jlens:
        continue
    h = res[l][0, last]
    lp = float(mx.softmax(readout(h).astype(mx.float32))[answer])
    jlogits = readout(transport(h, l)).astype(mx.float32)
    jp = float(mx.softmax(jlogits)[answer])
    mark = "    <--" if jp > 0.5 > lp else ""
    print(f"L{l:2d}    {lp:.3f}    {jp:.3f}    "
          f"{tokenizer.decode([int(mx.argmax(jlogits))])!r}{mark}")
```

'The capital of France is' -> ' Paris'

layer	logit p (ans)	jacobian p (ans)	jacobian top-1
L48	0.000	0.000	' _____ '
L49	0.000	0.004	' _____ '
L50	0.000	0.001	' _____ '
L51	0.000	0.001	' _____ '
L52	0.000	0.001	' _____ '
L53	0.000	0.012	' _____ '
L54	0.001	0.086	' _____ '
L55	0.088	0.895	' Paris' <--
L56	0.237	0.961	' Paris' <--
L57	0.520	0.972	' Paris'
L58	0.922	0.990	' Paris'
L59	0.856	0.987	' Paris'
L60	0.858	0.989	' Paris'
L61	0.940	0.980	' Paris'
L62	0.247	0.848	' Paris' <--

關鍵那一行是 **L55**:logit lens 只給 0.09,Jacobian lens 已經給 0.90。

同一條 residual、同一顆讀出頭,差別只在乘不乘 J_ℓ 。所以答案不是 L56 才成形的,而是**早就在那裡**,只是還沒轉到 logit lens 讀得到的方向上。這就是 Jacobian lens 的加值。

先做一個對照,不然這個結果不算數

公開 lens 在擬合時會跳過每條 **prompt** 的前 16 個 **position** ([issue #5](#) 就在講這件事)。而上面那條 prompt 只有 5 個 token,我們讀的是 position 4, 剛好落在 lens 沒擬合過的區間。

所以要補一個對照:在前面墊一段無關的話,把要讀的位置推到 16 以後,看結果還在不在。

```
In [12]: FILLER = ("Here are some notes about geography that provide background
context "
               "for the question that follows this introductory sentence. ")

def compare_lenses(prompt, layers=(50, 53, 54, 55, 56, 58)):
    ids = encode(prompt)
    res = residuals(ids)
    last = ids.shape[1] - 1
    ans = int(mx.argmax(readout(res[-1])[0, last].astype(mx.float32)))
    print(f"read position {last:>2}    answer {tokenizer.decode([ans])!r}")
    for l in layers:
        if l not in Jlens:
            continue
        h = res[l][0, last]
        lp = float(mx.softmax(readout(h).astype(mx.float32))[ans])
        jp = float(mx.softmax(readout(transport(h, l)).astype(mx.float32))
[ans])
        print(f"  L{l:2d}    logit {lp:.3f}    jacobian {jp:.3f}")

print("SHORT prompt, read position is inside the unfitted range")
compare_lenses("The capital of France is")
print("\nPADDED prompt, read position is inside the fitted range")
compare_lenses(FILLER + "The capital of France is")
```

SHORT prompt, read position is inside the unfitted range

read position 4 answer ' Paris'

L50	logit 0.000	jacobian 0.001
L53	logit 0.000	jacobian 0.012
L54	logit 0.001	jacobian 0.086
L55	logit 0.088	jacobian 0.895
L56	logit 0.237	jacobian 0.961
L58	logit 0.922	jacobian 0.990

PADDED prompt, read position is inside the fitted range

read position 23 answer ' Paris'

L50	logit 0.000	jacobian 0.217
L53	logit 0.001	jacobian 0.765
L54	logit 0.002	jacobian 0.796
L55	logit 0.117	jacobian 0.982
L56	logit 0.321	jacobian 0.979
L58	logit 0.970	jacobian 0.998

結果不但沒消失,反而更明顯。在有擬合過的位置上,L53 就已經是 logit 0.001 對 Jacobian 0.77, L50 也還有 0.000 對 0.22。

也就是說前面那個結論站得住,而且短 prompt 的版本其實是低估了兩把尺的落差。

4. 一個有趣的例子

用中文問同一件事,看模型在想什麼。


```
In [13]: PARIS = {11751, 57590, 109705} # ' Paris', 'Paris', and the Chinese token
```

```
def lens_trace(prompt, layers, k=3):
    ids = encode(prompt)
    res = residuals(ids)
    last = ids.shape[1] - 1
    print(f"{prompt!r}    output top-1 = "
          f"{tokenizer.decode([int(mx.argmax(readout(res[-1]))[0,
last])))]!r}")
    print("                jacobian top-3                logit top-3")
    for l in layers:
        if l not in Jlens:
            continue
        h = res[l][0, last]
        j = mx.argsort(-readout(transport(h, l)).astype(mx.float32))[:k]
        g = mx.argsort(-readout(h).astype(mx.float32))[:k]
        js = " ".join(f"{tokenizer.decode([int(t)])!r}" for t in j)
        gs = " ".join(f"{tokenizer.decode([int(t)])!r}" for t in g)
        print(f"  L{l:2d}  {js:40s}  {gs}")
```

```
lens_trace("法国的首都是", range(40, 62, 4))
```

	jacobian top-3	logit top-3
L40	' Paris' ' city' ' cities'	'...' ' _____' ' _____'
L44	' _____' ' city' ' cities'	'...' ' _____' '<u'
L48	' _____' ' _____' ' _____'	'<u' ' _____' ' _____'
L52	' ?' ' _____' ' _____'	' g`' ' 什么' ' 哪个'
L56	' 哪里' ' 哪个' ' Paris'	' 哪里' ' 哪个' ' 否'
L60	' Paris' ' 巴黎' ' Paris'	' 哪个' ' 哪里' ' 巴黎'

中文問句,但 Jacobian lens 在 **L40** 就把 ' Paris' 排在第一,一個英文 token。同一層的 logit lens 還只有 '...' 這種碎片。

到很後面 J-space 才轉出中文的 '巴黎',而模型表面上的下一個字是 '哪个',在忙著把句子接完(「法国的首都是哪个城市」)。

也就是:表面在造句,內部早就有答案了,而且答案先以英文的形式浮出來。這跟 Anthropic 在 [On the Biology of a Large Language Model](#) 描述的多語言迴路對得上,模型內部有一個不綁語言的概念空間,最後才轉成輸出語言。

順手比較幾個語言,看 Paris 最早在哪一層進到 top-5。

```
In [14]: def first_hit(prompt, targets, topk=5):
    ids = encode(prompt)
    res = residuals(ids)
    last = ids.shape[1] - 1
    fj = fl = None
    for l in range(n_layers):
        if l not in Jlens:
            continue
        h = res[l][0, last]
        j = set(int(t) for t in mx.argsort(-readout(transport(h,
l)).astype(mx.float32))[:topk].tolist()))
        g = set(int(t) for t in mx.argsort(-readout(h).astype(mx.float32))
[:topk].tolist()))
        if fj is None and j & targets:
            fj = l
        if fl is None and g & targets:
            fl = l
    return fj, fl

for p in ["The capital of France is", "法国的首都是",
        "La capitale de la France est", "フランスの首都は"]:
    fj, fl = first_hit(p, PARIS)
    gap = "" if None in (fj, fl) else f" {fl - fj} layers earlier"
    print(f"{p!r:34s} jacobian L{fj} logit L{fl}{gap}")
```

```
'The capital of France is' jacobian L54 logit L55 1
layers earlier
'法国的首都是' jacobian L23 logit L58 35
layers earlier
'La capitale de la France est' jacobian L21 logit L57 36
layers earlier
'フランスの首都は' jacobian L39 logit L56
17 layers earlier
```

中文和法文的落差最誇張:Jacobian lens 在 L21 到 L23 就把 Paris 排進前五, logit lens 要等到 L57 以後。

英文問句用這個指標反而看不出差距 (L54 對 L55),因為英文的 Paris 本來就不會太晚進 top-5。但看機率就很清楚,前面那張表的 L55 是 0.09 對 0.90。

合起來看:非英文問句在 residual stream 裡走的是「先想到答案,最後才翻成輸出語言」這條路,而 logit lens 那把尺只看得到翻譯完的最後階段。

5. 換掉 J-space 裡的概念

前面幾節都在讀。這一節反過來寫。

對齊原文的實驗:四個 prompt 分別問 France 的首都、語言、所屬洲、貨幣,在 J-space 裡把 France 換成 China,四個情境用完全相同的介入。如果四個答案一起改,代表它們讀的是同一份共享表徵。

做法是論文的 lens coordinate patching。每個 token 在 residual stream 裡有一個方向 v_t ,也就是 $W_U J_\ell$ 的第 t 行,讀出時的分數就是內積 $\langle v_t, h \rangle$ 。換概念是三步:

$$V = [v_s \ v_t], \quad c = V^\dagger h, \quad h \leftarrow h + \alpha V (\sigma(c) - c)$$

c 是 h 投影到這兩個方向上的座標, σ 把兩個座標交換。和 $\text{span}\{v_s, v_t\}$ 正交的成分完全不動,所以只有這一個概念被換掉。

$V^\dagger$ 是 pseudo-inverse。不能直接拿內積當座標, J-lens 向量 overcomplete 而且彼此不正交,下面會看到 v_{France} 跟 v_{China} 的 cosine 有 0.44, 直接取內積會把重疊的部分算兩次。

```
In [15]: lh = llm.lm_head
```

```
def unembed_row(token):
    i = mx.array([token])
    return mx.dequantize(lh.weight[i], lh.scales[i], lh.biases[i],
                        group_size=lh.group_size,
                        bits=lh.bits)[0].astype(mx.float32)

def jlens_vec(token, l):
    """The J-lens direction of one token: row t of W_U @ J_l."""
    return unembed_row(token) @ Jlens[l].astype(mx.float32)

def coords(x, vs, vt):
    V = mx.stack([vs, vt], axis=1)
    return V, mx.linalg.pinv(V, stream=mx.cpu) @ x

def swap(x, vs, vt, alpha):
    V, c = coords(x, vs, vt)
    return x + alpha * (V @ (c[:-1] - c))

def add_only(x, vs, vt, alpha):
    V, c = coords(x, vs, vt)
    return x + alpha * (c[0] - c[1]) * vt

FRANCE, CHINA = tokenizer.encode(" France")[0], tokenizer.encode(" China")
[0]
vf, vc = jlens_vec(FRANCE, 48), jlens_vec(CHINA, 48)
cos = float(vf @ vc / (mx.linalg.norm(vf) * mx.linalg.norm(vc)))
print(f"v_France at L48: {tuple(vf.shape)}, norm
{float(mx.linalg.norm(vf)):.1f}")
print(f"cos(v_France, v_China) = {cos:+.3f}")
```

```
v_France at L48: (5120,), norm 1.4
```

```
cos(v_France, v_China) = +0.441
```

三個選擇決定成不成:

- 層:L40 到 L55。前兩節看到這個 model 的 J-lens 讀出大約要到 L40 之後才穩定。
- 位置:只改 France 那個 token,不碰最後一個位置。最後幾層的 J-lens 方向是「準備要說出口的字」,在那裡動手 model 會直接把 China 說出來,而不是拿它去推理。
- 強度: $\alpha = 2$ 。 $\alpha = 1$ 是嚴格的交換,方向對但推不過去。

順便跑一個對照組 add_only :只加上 China 的成分,不把 France 拿掉。

```
In [16]: BAND = [l for l in Jlens if 40 <= l < 56]
ALPHA = 2.0
```

```
def patched(ids, pos, alpha, edit=swap):
    h = inner.embed_tokens(ids)
    fa, ssm, p = prep(h)
    for l, layer in enumerate(inner.layers):
        h = layer(h, mask=(ssm if layer.is_linear else fa),
                    cache=None, position_ids=p, position_embeddings=None)
    if alpha and l in BAND:
        h[0, pos] = edit(h[0, pos].astype(mx.float32),
                          jlens_vec(FRANCE, l), jlens_vec(CHINA, l),
                          alpha).astype(dtype)
    return readout(h)[0, ids.shape[1] - 1].astype(mx.float32)

def p_of(sc, s):
    return float(mx.softmax(sc)[tokenizer.encode(s)[0]])

def top1(sc):
    return tokenizer.decode([int(mx.argmax(sc))])

TASKS = [("The capital of France is", " Paris", " Beijing"),
          ("The language spoken in France is", " French", " Chinese"),
          ("The continent that contains France is", " Europe", " Asia"),
          ("The currency of France is called the", " Euro", " Yuan")]

for prompt, orig, new in TASKS:
    ids = encode(prompt)
    toks = [tokenizer.decode([int(t)]) for t in ids[0].tolist()]
    pos = next(i for i, t in enumerate(toks) if "France" in t)
    print(f"{prompt!r}")
    for tag, sc in [("before ", patched(ids, pos, 0)),
                    ("swap    ", patched(ids, pos, ALPHA)),
                    ("add only", patched(ids, pos, ALPHA, add_only))]:
        print(f"    {tag} {top1(sc)!r:11s} p({orig})={p_of(sc, orig):.3f}"
              f"    p({new})={p_of(sc, new):.3f}")
```

'The capital of France is'

before	' Paris'	p(Paris)=0.617	p(Beijing)=0.000
swap	' Beijing'	p(Paris)=0.006	p(Beijing)=0.383
add only	' Paris'	p(Paris)=0.427	p(Beijing)=0.066

'The language spoken in France is'

before	' French'	p(French)=0.539	p(Chinese)=0.000
swap	' Chinese'	p(French)=0.004	p(Chinese)=0.222
add only	' French'	p(French)=0.461	p(Chinese)=0.002

'The continent that contains France is'

before	' Europe'	p(Europe)=0.153	p(Asia)=0.001
swap	' Asia'	p(Europe)=0.090	p(Asia)=0.096
add only	' Europe'	p(Europe)=0.138	p(Asia)=0.005

'The currency of France is called the'

before	' euro'	p(Euro)=0.111	p(Yuan)=0.000
--------	---------	----------------	----------------

```
swap      ' yuan'      p( Euro)=0.001  p( Yuan)=0.060
add only  ' euro'      p( Euro)=0.097  p( Yuan)=0.036
```

四個問題,同一個介入,答案全部跟著改:Paris 變 Beijing、French 變 Chinese、Europe 變 Asia、euro 變 yuan。add only 對照組四個都沒翻,所以「把 France 拿掉」這一半是必要的,不是單純注入 China 就能達到。

要留意幅度。嚴格交換($\alpha = 1$)只推得動一點點, $p(\text{Beijing})$ 從 0.000 到 0.034, 要 $\alpha = 2$ 才翻得過去。Neel Nanda 團隊在同一個 model 上複現時也只拿到微弱但為正的因果效果。這是一個 27B 的 4-bit 量化 model,不是 Sonnet 4.5, 效應本來就弱得多,要看的是方向對不對。Asia 那一列最勉強,0.096 對 Europe 0.090, 剛好翻過去而已。

小結

- **residual stream** 是所有層共用的座標系,所以才有得讀。
- **logit lens** 把最後那顆頭借來讀中間層,便宜,但暗中假設 $J_\ell = I$ 。
- **Jacobian lens** 把 J_ℓ 真的算出來,讀同一條 residual 卻早好幾層看到答案。
- 從零算 J_ℓ 只需要 `mx.vjp` 加兩個實務 workaround,一條 prompt 的結果就跟一千條 擬合的公開 lens 高度對齊。

該保留的懷疑

- **線性近似**。 J_ℓ 是一階展開,後面幾十層並不是線性的。
- **全域先驗**。公開 lens 是跨一千條 prompt 平均的,會帶進跟當前輸入無關的偏好。
- **input-copying**。如果要探測的字本來就出現在 prompt 裡,lens 會在那個位置直接把它讀成第一名,那反映的是輸入,不是「內部想法」。上面的例子沒有踩到這點(Paris 不在 prompt 裡),但要自己延伸實驗時得檢查(issue #5)。
- **名字取得不好**。「J-space」並不是 residual stream 的一個線性子空間。照論文自己的 methods,它是「在稀疏限制下的一堆多面錐的聯集」。
- **循環論證的疑慮**。一個依「對最終輸出的影響」定義出來的空間,本來就會跟最終輸出高度相關。
- **不是每個實驗都能重現**。獨立重現裡,多步事實的替換站得住,但詩韻規劃和心算沒有重現出來(Nanda 團隊的 review、tao-hpu/jspace-replication)。

參考資料

- [anthropics/jacobian-lens](#) 官方實作
- [neuronpedia/jacobian-lens](#) 擬合好的 lens
- [Interpreting GPT: the logit lens](#)
- [A Mathematical Framework for Transformer Circuits](#)
- [On the Biology of a Large Language Model](#)
- [J-Space: Yet Another LLM Mind Reader?](#)
- [A Review of Anthropic's Global Workspace Paper](#)
- [Gated Delta Networks](#)